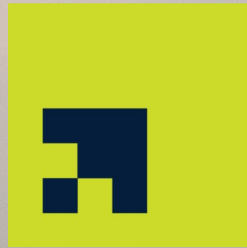


Exception Handling

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences





Errors and Exceptions

- Someone needs to know more about errors
 - When something goes wrong with a computer program, what could be the reason?
- The program should inform
 - Other parts of the program
 - And potentially users running the program
- One way is by generating an error object and propagating it through the code until either
 - Something handles the error and sorts things out
 - The point at which the program is entered is found



Errors and Exceptions

- The user who ran the program needs to know that something has gone wrong

- If the Error propagates out of the program
- They are notified of a problem via a short report on the Error that occurred and a stack trace of where that error can be found
- For example, the following screen dump illustrates a programming error
 - Where someone has tried to concatenate a string and a number together using the '+' operator
 - This error has propagated out of the program and a stack trace of the code that was called is presented

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_23075/3476216514.py in <module>  
      9  
     10 p = Person("John", 20)  
----> 11 print(p)  
  
/tmp/ipykernel_23075/3476216514.py in __str__(self)  
      5  
      6     def __str__(self):  
----> 7         return "This is " + self.name + ". I am " + self.age + ".  
      8  
      9  
  
TypeError: can only concatenate str (not "int") to str
```



Errors and Exceptions

[Hunt 2019]

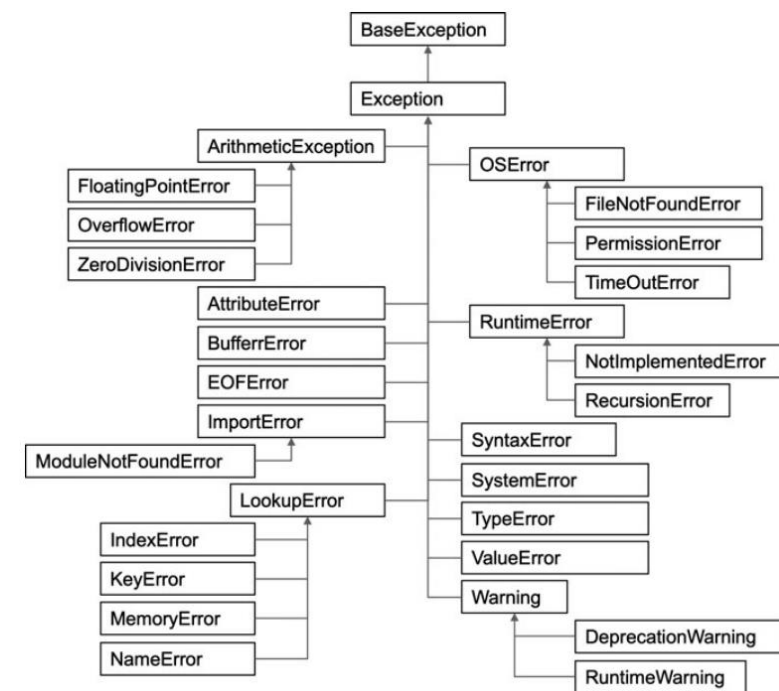
4

- In Python, the terms Error and Exception are used interchangeably

- Although from a style point of view
 - Exceptions might be used to represent issues with operations, such as arithmetic exceptions
 - Errors might be associated with functional issues such as a file not being found

- Exception/Error is an object

- Like everything in Python
- We call it “Handler”, an object to handle an error
- The Exception/Error types are defined in a class hierarchy
 - With the root of this hierarchy being the BaseException type
 - All built-in errors and exceptions eventually extend from the BaseException type
 - It has a subclass Exception which is the root of all user defined exceptions
 - As well as many built-in exceptions





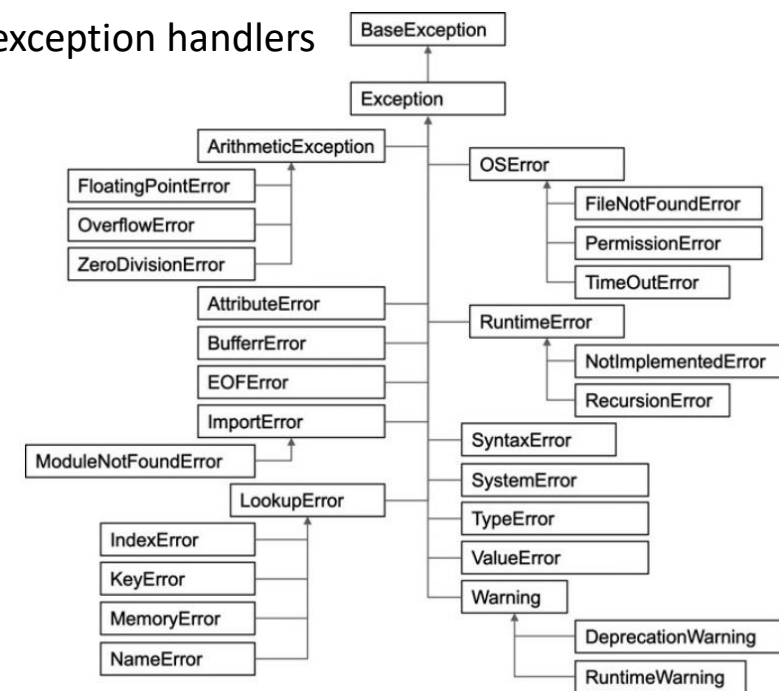
Exception Handling

[Hunt 2019]

5

- Different types of error produce different exception

- For example,
 - if the error is caused by dividing an integer by zero, then the exception is an arithmetic exception
- The type of exception is identified by objects and can be caught and processed by exception handlers
- Each exception handler can deal with exceptions associated with its class of error





Exception Handling

- The purpose of exception handling, is to handle an error condition when it happens at run time
 - System does not allow an error to go unnoticed
 - For example, if we try to divide by zero
 - The system generates an error and stops execution to prevent unusual behavior
 - It may mean that the user has entered an incorrect value, or a previous error propagated
 - We use exceptions to force the user to correct the mistake and rerun the calculation or drive the execution safely
- An exception moves the flow of control from one place to another
 - In most situations, this is because a problem occurs which cannot be handled locally
 - But that can be handled in another part of the system



Exception Handling Terminology

- Exception
 - A runtime error
- Raising an exception
 - Generating a new exception
- Throwing an exception
 - Triggering a generated exception
- Handling an exception
 - Processing code that deals with the error
- Handler
 - The code that deals with the error, called the `catch` block



Handling an Exception

• Catch an exception using `try-except` construct

- `try` block
 - The try block indicates the code that should be executed and monitored for the exceptions listed in the except expressions
- `except` clause
 - You can use an optional except clause to indicate what to do when certain classes of exception/error occur
 - E.g., resolve the problem or generate a warning message
 - There can be any number of `except clauses in sequence` checking for different types of error/exceptions
- `else` clause
 - This is an optional clause which will be run if and only if no exception was thrown in the try block
 - It is useful for code that must be executed if the try clause does not raise an exception
- `finally` clause
 - The optional finally clause runs after the try block exits
 - Whether or not this is due to an exception being raised
 - You can use it to clean up any resources, close files, etc

```
try:
    <codes>
except <exception type>:
    <codes>
else:
    <codes>
finally:
    <codes>
```




Handling an Exception

- Consider the following function which divides a number by zero

- Symantec error; divide by zero
- This should raise the ZeroDivisionError and block the execution
- If we now call this function, we will get the error traceback in the standard output

```
def runcalc(x):
```

x / 0

```
runcalc(6)
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
/tmp/ipykernel_23075/1298772460.py in <module>  
----> 1 runcalc(6)  
  
/tmp/ipykernel_23075/1434401507.py in runcalc(x)  
      1 def runcalc(x):  
----> 2     x / 0  
  
ZeroDivisionError: division by zero
```



Handling an Exception

- To handle this exception

- Wrap the call of “runcalc” within a try statement and provide an except clause
- It now results in the string 'oops' being printed out instead of an error
 - When “runcalc” is called, the '/' operator throws the ZeroDivisionError
 - It is passed back to the calling code which has an except clause specifying this type of exception
 - This catches the exception and runs the associated code block

```
try:  
    runcalc(6)  
except ZeroDivisionError:  
    print("oops")
```

```
try:  
    runcalc(6)  
except ZeroDivisionError:  
    print("oops")
```

oops



Handling an Exception

- You can define different behaviours for different exceptions

- If you do not want to have a common block of code handling your exceptions
- This is done by having a series of except clauses
- if different errors could happen
 - Each monitor has a different type of exception
- The except clause can be given a class Exception to check for
 - It will match any exception of the existing subclass of the exception
 - The Exception class is a grandparent of the ZeroDivisionError
- As only one except clause is allowed to run
 - If the final except handler came first, the other excepts will never ever be run

```
try:  
    runcalc(6)  
except ZeroDivisionError:  
    print('oops')  
except IndexError:  
    print('arrgh')  
except FileNotFoundError:  
    print('huh!')  
except Exception:  
    print('Duh!')
```



Handling an Exception

- We do not have to be as precise as this
 - It is also possible to specify an except clause that can be used to catch any type of error or exception
 - This must be the last except clause as it omits the exception type and thus acts as a wildcard
 - It can be used to ensure that you get notified that an error occurred
 - You do not know what type of error it was
 - Use it feature with caution

```
try:  
    runcalc(6)  
except Exception:  
    print("oops")
```

```
try:  
    runcalc(6)  
except:  
    print("oops")
```



Accessing the Exception Object

- You can access the exception object using the **except** clause

- Using the **“as”** keyword
- If there are multiple except clauses
 - each except clause can decide to bind the exception object to a variable or not
 - Each variable can have a different name

```
try:  
    runcalc(6)  
except ZeroDivisionError as exp:  
    print(exp)  
    print("oops")
```

```
division by zero  
oops
```




Jumping to Exception Handlers

- When an Error/Exception is raised, it is immediately thrown to the exception handlers

- Go to the except clauses
- Any statements following the raised exception are not executed
- The term throwing is used when error or exception is raised, it throws execution to the handled part
 - Or it is thrown out of the application if no except clause is found to handle the error/exception

```
def my_function(x, y):  
    print("my_function in")  
    result = x / y  
    print("my_function out")  
    return result
```

Y != 0

```
try:  
    print("Before my_function")  
    my_function(6, 2)  
    print("After my_function")  
except ZeroDivisionError as exp:  
    print("oops")
```

Before my_function
my_function in
my_function out
After my_function

Y == 0

```
try:  
    print("Before my_function")  
    my_function(6, 0)  
    print("After my_function")  
except ZeroDivisionError as exp:  
    print("oops")
```

Before my_function
my_function in
oops



The Else Clause

- Optional else clause is to be present after all except clauses
 - The else clause is executed if and only if no exceptions were raised

```
try:  
    my_function(6, 2)  
except ZeroDivisionError as e:  
    print(e)  
else:  
    print("Everything worked OK")
```

```
my_function in  
my_function out  
Everything worked OK
```

```
try:  
    my_function(6, 0)  
except ZeroDivisionError as e:  
    print(e)  
else:  
    print("Everything worked OK")
```

```
my_function in  
division by zero
```



The Finally Clause

[Hunt 2019]

17

- An optional finally clause is the last clause in the statement

- It comes after all except clauses and the else clause
- It is used for code that you want to run whether an exception occurred or not
 - Such as shutting down or closing any resources that your code might be using

```
try:
    my_function(6, 2)
except ZeroDivisionError as e:
    print(e)
else:
    print("Everything worked OK")
finally:
    print("Always runs")
```

```
my_function in
my_function out
Everything worked OK
Always runs
```

```
try:
    my_function(6, 0)
except ZeroDivisionError as e:
    print(e)
else:
    print("Everything worked OK")
finally:
    print("Always runs")
```

```
my_function in
division by zero
Always runs
```



Raising an Exception

- An error or exception can be raised by the user using the keyword “raise”

- In the example, the second statement in the function body will create a new instance of the “ValueError” class and then raise it
 - So that it is thrown allowing it to be caught by any exception handlers that have been defined
- We can handle this exception
 - By writing a try block with an except clause for the ValueError class

`raise <Exception/Error type to raise>()`

```
def function_bang():  
    print("function_bang in")  
    raise ValueError("Bang!")  
    print("function_bang")
```

```
try:  
    function_bang()  
except ValueError as ve:  
    print(ve)
```

```
function_bang in  
Bang!
```



Raising an Exception

- You can just provide the name of the exception class to the raise keyword
 - If you just want to raise an exception without providing any constructor arguments

```
raise ValueError # short hand for raise ValueError()
```



Defining a Custom Exception

• You can define your own Errors and Exceptions

- Which can give you more control over what happens in particular circumstances
- To define an exception, you create a subclass of the Exception class or one of its subclasses
- For example, let us define a InvalidAgeException
 - We can extend the Exception class and generate an appropriate message
 - This class can be used to explicitly represent an issue when an age is set on a Person which is not within the acceptable age range
 - We provide an initialiser to allow parameters to be passed into the new instance of the InvalidAgeException
 - We have also defined a suitable `__str__()` method to convert the exception into a string for printing purposes

```
class InvalidAgeException(Exception):  
    """ Valid Ages must be between 0 and 120 """  
  
    def __init__(self, value):  
        self.value = value  
  
    def __str__(self):  
        return "InvalidAgeException(" + str(self.value) + ")"
```




Defining a Custom Exception

- We can capture the fact that an invalid age has been specified

- If the exception is raised a message will be printed out giving the actual value that caused the problem

```
class Person:
```

```
    def __init__(self, name, age):  
        self.set_name(name)  
        self.set_age(age)
```

```
    def set_name(self, value):  
        self.name = value
```

```
    def set_age(self, value):  
        if isinstance(value, int) and (value > 0 and value < 120):  
            self.age = value  
        else:  
            raise InvalidAgeException(value)
```

```
p = Person("John", -20)
```

```
-----  
InvalidAgeException                                Traceback (most recent call last)  
/tmp/ipykernel_23075/2410753552.py in <module>  
----> 1 p = Person("John", -20)  
  
/tmp/ipykernel_23075/3356575433.py in __init__(self, name, age)  
      2     def __init__(self, name, age):  
      3         self.set_name(name)  
----> 4         self.set_age(age)  
      5  
      6     def set_name(self, value):  
  
/tmp/ipykernel_23075/3356575433.py in set_age(self, value)  
     11         self.age = value  
     12     else:  
----> 13         raise InvalidAgeException(value)  
  
InvalidAgeException: InvalidAgeException(-20)
```